Shirley Zamora

Part One | Youtube Tutorial |

When opening OpenSSL, the user has to start with openssl because it is the program we are running, and the rest is the command. This is different from other languages where you are already inside the program. The cipher commands in OpenSSL are used to work with encryption methods, mostly symmetric ciphers, which use one key to both encrypt and decrypt data. When creating a message, nano is used to create files to where the message will live, and cat is used to display and read messages from the files created. When encrypting a message with OpenSSL, the user starts with openssl and uses the enc command, which does the symmetric encryption. The user can choose the encryption algorithm (like AES), the key size (256 bits), and the mode of operation (like CBC), which determines how the data is encrypted. Which is then followed by the binary to text encoding format (Base64). The program asks for a password, which will then be used to learn the encryption key. I also learned that when decrypting the only thing that changed is the -d, after that it prompts the password that was set in the beginning before it decrypts the message. I also learned how to use the RSA asymmetric encryption (two keys instead of one), and that the key size affected the security of the encryption. The larger it is the harder it is to break, but it is also slower to use. When displaying key information, it began with the modulus (n), which is part of both private and public keys. The public exponent (e) and the private exponent (d). RSA is also based on large prime numbers used to generate these values. RSA ensures that only the receiver can decrypt it using the private key.

|Tasks|

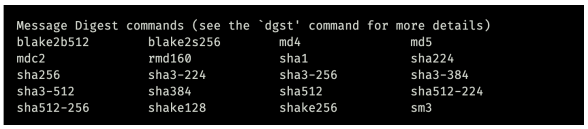
1: A,B,C

```
Usage: openssl [command] [params]

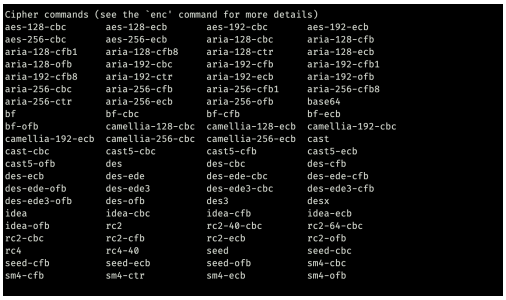
$ openssl
```

```
help:

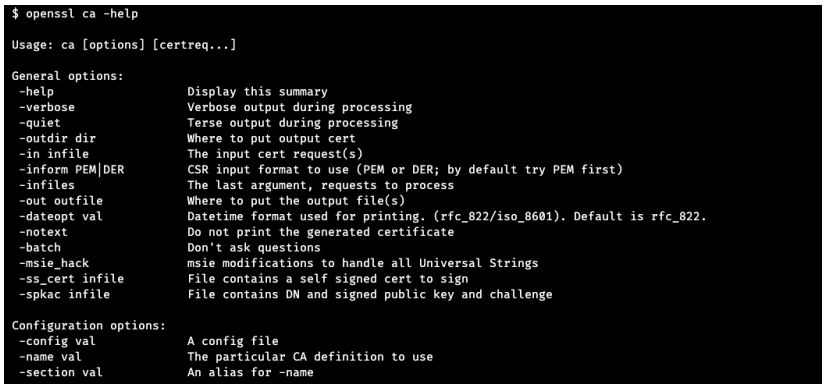
Standard commands
asn1parse         ca              ciphers         cmp
cms               configutl       crl             crl2pkcs7
dgst              dhparam         dsa             dsaparam
ec                ecparam         enc             errstr
fipsinstall       gendsa          genpkey         genrsa
help              info            kdf             list
mac               nseq            ocsp            passwd
pkcs12            pkcs7           pkcs8           pkey
pkeyparam         pkeyutl         prime           rand
rehash            req             rsa             rsautl
s_client          s_server        s_time          sess_id
skeyutl           smime           speed           spkac
srp               storeutl        ts              verify
version           x509
```



```
Message Digest commands (see the `dgst' command for more details)
blake2b512        blake2s256        md4               md5
mdc2              rmd160            sha1              sha224
sha256            sha3-224          sha3-256          sha3-384
sha3-512          sha384            sha512            sha512-224
sha512-256        shake128          shake256          sm3
```



```
Cipher commands (see the `enc' command for more details)
aes-128-cbc       aes-128-ecb       aes-192-cbc       aes-192-ecb
aes-256-cbc       aes-256-ecb       aria-128-cbc      aria-128-cfb
aria-128-cfb1     aria-128-cfb8     aria-128-ctr      aria-128-ecb
aria-128-ofb      aria-192-cbc      aria-192-cfb      aria-192-cfb1
aria-192-cfb8     aria-192-ctr      aria-192-ecb      aria-192-ofb
aria-256-cbc      aria-256-cfb      aria-256-cfb1     aria-256-cfb8
aria-256-ctr      aria-256-ecb      aria-256-ofb      base64
bf                bf-cbc            bf-cfb            bf-ecb
bf-ofb            camellia-128-cbc  camellia-128-ecb  camellia-192-cbc
camellia-192-ecb  camellia-256-cbc  camellia-256-ecb  cast
cast-cbc          cast5-cbc         cast5-cfb         cast5-ecb
cast5-ofb         des               des-cbc           des-cfb
des-ecb           des-ede           des-ede-cbc       des-ede-cfb
des-ede-ofb       des-ede3          des-ede3-cbc      des-ede3-cfb
des-ede3-ofb      des-ofb           des3              desx
idea              idea-cbc          idea-cfb          idea-ecb
idea-ofb          rc2               rc2-40-cbc        rc2-64-cbc
rc2-cbc           rc2-cfb           rc2-ecb           rc2-ofb
rc4               rc4-40            seed              seed-cbc
seed-cfb          seed-ecb          seed-ofb          sm4-cbc
sm4-cfb           sm4-ctr           sm4-ecb           sm4-ofb
```



```
$ openssl ca -help

Usage: ca [options] [certreq...]

General options:
 -help                  Display this summary
 -verbose               Verbose output during processing
 -quiet                 Terse output during processing
 -outdir dir            Where to put output cert
 -in infile             The input cert request(s)
 -inform PEM|DER        CSR input format to use (PEM or DER; by default try PEM first)
 -infiles               The last argument, requests to process
 -out outfile           Where to put the output file(s)
 -dateopt val           Datetime format used for printing. (rfc_822/iso_8601). Default is rfc_822.
 -notext                Do not print the generated certificate
 -batch                 Don't ask questions
 -msie_hack             msie modifications to handle all Universal Strings
 -ss_cert infile        File contains a self signed cert to sign
 -spkac infile          File contains DN and signed public key and challenge

Configuration options:
 -config val            A config file
 -name val              The particular CA definition to use
 -section val           An alias for -name
```

When writing the OpenSSL in the terminal, the different commands appear, starting with the standard commands, then the Digest commands and then the cipher commands. When using the list command to list each of these command types, I struggled to figure out how to use the list command, as I would type in the terminal *list -commands* to which this error would appear: [ERROR] TypeError: Failed to fetch. It wasn't until I added the openssl in front of *list -commands* that it would show me just the standard commands. The Standard commands deal with generating keys, and certificates. When using *openssl list -cipher-commands,* the cipher commands appear which are used to encrypt and decrypt data using symmetric encryption (one key). When using the *openssl list -digest-commands* the digest commands appear which are used to verify the values of data and ensure integrity. When using the help command, I was struggled to figure out how to use the help command that is because I was not adding the dash before the help command: *Using configuration from /usr/local/ssl/openssl.cnf*
variable lookup failed for ca::default_ca
00000000:error:0700006C:configuration file routines:NCONF_get_string:no value:crypto/conf/conf_lib.c:316:group=ca name=default_ca, when adding the dash I was able to get an in depth list of what the command is able to do.

2) A,B

```
$ openssl speed

Doing mdc2 ops for 3s on 16 size blocks: 2985546 mdc2 ops in 0.00s
Doing mdc2 ops for 3s on 64 size blocks: 890039 mdc2 ops in 0.00s
Doing mdc2 ops for 3s on 256 size blocks: 232478 mdc2 ops in 0.00s
Doing mdc2 ops for 3s on 1024 size blocks: 58142 mdc2 ops in 0.00s
Doing mdc2 ops for 3s on 8192 size blocks: 7328 mdc2 ops in 0.00s
Doing mdc2 ops for 3s on 16384 size blocks: 1349 mdc2 ops in 0.00s
Doing md4 ops for 3s on 16 size blocks: 2589017 md4 ops in 0.00s
Doing md4 ops for 3s on 64 size blocks: 4953241 md4 ops in 0.00s
Doing md4 ops for 3s on 256 size blocks: 3231583 md4 ops in 0.00s
Doing md4 ops for 3s on 1024 size blocks: 651342 md4 ops in 0.00s
Doing md4 ops for 3s on 8192 size blocks: 101873 md4 ops in 0.00s
Doing md4 ops for 3s on 16384 size blocks: 67634 md4 ops in 0.00s
Doing md5 ops for 3s on 16 size blocks: 2322235 md5 ops in 0.00s
Doing md5 ops for 3s on 64 size blocks: 1898105 md5 ops in 0.00s
Doing md5 ops for 3s on 256 size blocks: 1281417 md5 ops in 0.00s
Doing md5 ops for 3s on 1024 size blocks: 614612 md5 ops in 0.00s
Doing md5 ops for 3s on 8192 size blocks: 78947 md5 ops in 0.00s
Doing md5 ops for 3s on 16384 size blocks: 39395 md5 ops in 0.00s
Doing sha1 ops for 3s on 16 size blocks: 2021884 sha1 ops in 0.00s
```

```
Doing sha1 ops for 3s on 64 size blocks: 1865616 sha1 ops in 0.00s
Doing sha1 ops for 3s on 256 size blocks: 1158449 sha1 ops in 0.00s
Doing sha1 ops for 3s on 1024 size blocks: 437232 sha1 ops in 0.00s
Doing sha1 ops for 3s on 8192 size blocks: 65566 sha1 ops in 0.00s
Doing sha1 ops for 3s on 16384 size blocks: 35669 sha1 ops in 0.00s
Doing sha256 ops for 3s on 16 size blocks: 1559293 sha256 ops in 0.00s
Doing sha256 ops for 3s on 64 size blocks: 1228530 sha256 ops in 0.00s
Doing sha256 ops for 3s on 256 size blocks: 629861 sha256 ops in 0.00s
Doing sha256 ops for 3s on 1024 size blocks: 215437 sha256 ops in 0.00s
Doing sha256 ops for 3s on 8192 size blocks: 32677 sha256 ops in 0.00s
Doing sha256 ops for 3s on 16384 size blocks: 14862 sha256 ops in 0.00s
Doing sha512 ops for 3s on 16 size blocks: 1332404 sha512 ops in 0.00s
Doing sha512 ops for 3s on 64 size blocks: 1181501 sha512 ops in 0.00s
Doing sha512 ops for 3s on 256 size blocks: 865116 sha512 ops in 0.00s
Doing sha512 ops for 3s on 1024 size blocks: 268588 sha512 ops in 0.00s
Doing sha512 ops for 3s on 8192 size blocks: 40546 sha512 ops in 0.00s
Doing sha512 ops for 3s on 16384 size blocks: 22163 sha512 ops in 0.00s
Doing whirlpool ops for 3s on 16 size blocks: 905509 whirlpool ops in 0.00s
Doing whirlpool ops for 3s on 64 size blocks: 542257 whirlpool ops in 0.00s
Doing whirlpool ops for 3s on 256 size blocks: 286113 whirlpool ops in 0.00s
Doing whirlpool ops for 3s on 1024 size blocks: 82271 whirlpool ops in 0.00s
```

```
Doing whirlpool ops for 3s on 8192 size blocks: 11698 whirlpool ops in 0.00s
Doing whirlpool ops for 3s on 16384 size blocks: 5574 whirlpool ops in 0.00s
Doing rmd160 ops for 3s on 16 size blocks: 1981786 rmd160 ops in 0.00s
Doing rmd160 ops for 3s on 64 size blocks: 1455369 rmd160 ops in 0.00s
Doing rmd160 ops for 3s on 256 size blocks: 736931 rmd160 ops in 0.00s
Doing rmd160 ops for 3s on 1024 size blocks: 281086 rmd160 ops in 0.00s
Doing rmd160 ops for 3s on 8192 size blocks: 44094 rmd160 ops in 0.00s
Doing rmd160 ops for 3s on 16384 size blocks: 22188 rmd160 ops in 0.00s
Doing hmac(sha256) ops for 3s on 16 size blocks: 1246667 hmac(sha256) ops in 0.00s
Doing hmac(sha256) ops for 3s on 64 size blocks: 1035960 hmac(sha256) ops in 0.00s
Doing hmac(sha256) ops for 3s on 256 size blocks: 570076 hmac(sha256) ops in 0.00s
Doing hmac(sha256) ops for 3s on 1024 size blocks: 219310 hmac(sha256) ops in 0.00s
Doing hmac(sha256) ops for 3s on 8192 size blocks: 31640 hmac(sha256) ops in 0.00s
Doing hmac(sha256) ops for 3s on 16384 size blocks: 15616 hmac(sha256) ops in 0.00s
Doing des-cbc ops for 3s on 16 size blocks: 2971429 des-cbc ops in 0.00s
Doing des-cbc ops for 3s on 64 size blocks: 1410142 des-cbc ops in 0.00s
Doing des-cbc ops for 3s on 256 size blocks: 405080 des-cbc ops in 0.00s
Doing des-cbc ops for 3s on 1024 size blocks: 97004 des-cbc ops in 0.00s
Doing des-cbc ops for 3s on 8192 size blocks: 12026 des-cbc ops in 0.00s
Doing des-cbc ops for 3s on 16384 size blocks: 6192 des-cbc ops in 0.00s
Doing des-ede3 ops for 3s on 16 size blocks: 1656336 des-ede3 ops in 0.00s
```

```
Doing des-ede3 ops for 3s on 64 size blocks: 501363 des-ede3 ops in 0.00s
Doing des-ede3 ops for 3s on 256 size blocks: 138313 des-ede3 ops in 0.00s
Doing des-ede3 ops for 3s on 1024 size blocks: 36099 des-ede3 ops in 0.00s
Doing des-ede3 ops for 3s on 8192 size blocks: 4527 des-ede3 ops in 0.00s
Doing des-ede3 ops for 3s on 16384 size blocks: 2262 des-ede3 ops in 0.00s
Doing aes-128-cbc ops for 3s on 16 size blocks: 3910223 aes-128-cbc ops in 0.00s
Doing aes-128-cbc ops for 3s on 64 size blocks: 2076678 aes-128-cbc ops in 0.00s
Doing aes-128-cbc ops for 3s on 256 size blocks: 733862 aes-128-cbc ops in 0.00s
Doing aes-128-cbc ops for 3s on 1024 size blocks: 197679 aes-128-cbc ops in 0.00s
Doing aes-128-cbc ops for 3s on 8192 size blocks: 25741 aes-128-cbc ops in 0.00s
Doing aes-128-cbc ops for 3s on 16384 size blocks: 13156 aes-128-cbc ops in 0.00s
Doing aes-192-cbc ops for 3s on 16 size blocks: 3621590 aes-192-cbc ops in 0.00s
Doing aes-192-cbc ops for 3s on 64 size blocks: 1867853 aes-192-cbc ops in 0.00s
Doing aes-192-cbc ops for 3s on 256 size blocks: 630796 aes-192-cbc ops in 0.00s
Doing aes-192-cbc ops for 3s on 1024 size blocks: 174628 aes-192-cbc ops in 0.00s
Doing aes-192-cbc ops for 3s on 8192 size blocks: 22507 aes-192-cbc ops in 0.00s
Doing aes-192-cbc ops for 3s on 16384 size blocks: 11405 aes-192-cbc ops in 0.00s
Doing aes-256-cbc ops for 3s on 16 size blocks: 3429572 aes-256-cbc ops in 0.00s
Doing aes-256-cbc ops for 3s on 64 size blocks: 1683187 aes-256-cbc ops in 0.00s
Doing aes-256-cbc ops for 3s on 256 size blocks: 567702 aes-256-cbc ops in 0.00s
Doing aes-256-cbc ops for 3s on 1024 size blocks: 155624 aes-256-cbc ops in 0.00s
```

When running the speed command, I noticed it began to test all the commands available in OpenSSL, meaning that my computer was never going to be able to test them all as it was testing it slowly one by one.

```
$ openssl speed rsa2048

Doing 2048 bits private rsa sign ops for 10s: 3668 2048 bits private RSA sign ops in 0.00s
Doing 2048 bits public rsa verify ops for 10s: 133443 2048 bits public RSA verify ops in 0.00s
Doing 2048 bits public rsa encrypt ops for 10s: 127836 2048 bits public RSA encrypt ops in 0.00s
Doing 2048 bits private rsa decrypt ops for 10s: 3601 2048 bits private RSA decrypt ops in 0.00s
Doing rsa2048 keygen ops for 10s: 51 rsa2048 KEM keygen ops in 0.00s
Doing rsa2048 encaps ops for 10s: 123760 rsa2048 KEM encaps ops in 0.00s
Doing rsa2048 decaps ops for 10s: 3584 rsa2048 KEM decaps ops in 0.00s
Doing rsa2048 keygen ops for 10s: 58 rsa2048 signature keygen ops in 0.00s
Doing rsa2048 signs ops for 10s: 3649 rsa2048 signature sign ops in 0.00s
Doing rsa2048 verify ops for 10s: 132177 rsa2048 signature verify ops in 0.00s
version: 3.6.1
built on: Wed Mar 11 12:20:20 2026 UTC
options: bn(64,64)
compiler: /emsdk/upstream/emscripten/emcc -mx32 -Wall -O3 -DOPENSSL_USE_NODELETE -DL_ENDIAN -DOPENSSL_BUILDING_OPENSSL -DNDEBUG
CPUINFO: N/A
                  sign    verify    encrypt   decrypt   sign/s verify/s  encr./s  decr./s
rsa  2048 bits 0.000000s 0.000000s 0.000000s 0.000000s      inf      inf      inf      inf
                            keygen    encaps    decaps keygens/s  encaps/s  decaps/s
                     rsa2048 0.000000s 0.000000s 0.000000s       inf       inf       inf
                            keygen     signs    verify keygens/s    sign/s  verify/s
                     rsa2048 0.000000s 0.000000s 0.000000s       inf       inf       inf
```

```
$ openssl speed aes-128-cbc

Doing aes-128-cbc ops for 3s on 16 size blocks: 14939378 aes-128-cbc ops in 0.00s
Doing aes-128-cbc ops for 3s on 64 size blocks: 7724542 aes-128-cbc ops in 0.00s
Doing aes-128-cbc ops for 3s on 256 size blocks: 2319996 aes-128-cbc ops in 0.00s
Doing aes-128-cbc ops for 3s on 1024 size blocks: 191915 aes-128-cbc ops in 0.00s
Doing aes-128-cbc ops for 3s on 8192 size blocks: 23496 aes-128-cbc ops in 0.00s
Doing aes-128-cbc ops for 3s on 16384 size blocks: 12801 aes-128-cbc ops in 0.00s
version: 3.6.1
built on: Wed Mar 11 12:20:20 2026 UTC
options: bn(64,64)
compiler: /emsdk/upstream/emscripten/emcc -mx32 -Wall -O3 -DOPENSSL_USE_NODELETE -DL_ENDIAN -DOPENSSL_BUILDING_OPENSSL -DNDEBUG
CPUINFO: N/A
The 'numbers' are in 1000s of bytes per second processed.
type             16 bytes     64 bytes    256 bytes   1024 bytes   8192 bytes  16384 bytes
aes-128-cbc           infk         infk         infk         infk         infk         infk
```

B) When comparing both speed signatures AES was a lot faster than RSA, that is because AES encrypts large amounts of data very quickly as it uses simpler operations. RSA is slower because they can handle a small number of operations rather than in bulk like AES. AES is better at encrypting rather than RSA.

3) A: i, ii,iii

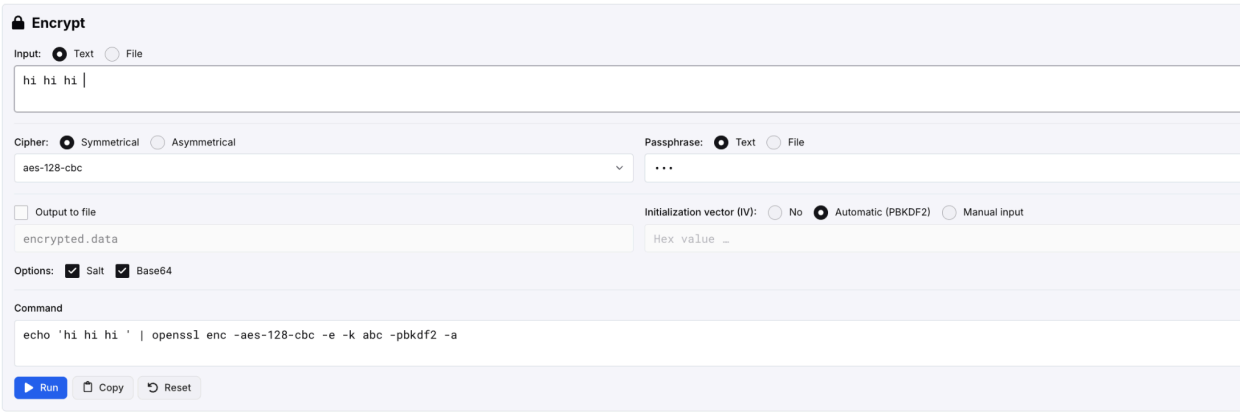


Throughout this project I was working with the web IDE, so when trying to use Nano to create the file, the web IDE wasn't able to recognize the command directly from its terminal.

```
Usage: openssl [command] [params]

$ echo "hi hi hi" > file.txt

[WARN] Advanced bash features are not supported! Only the pipe '|' works for now.
hi hi hi
```



However, the web IDE did have at the bottom an option to create the file through its tabs, when creating the file *(file.txt),* I was then able to use the same tool to encrypt the message in the file *(hi hi hi)*. The enc is the symmetric encryption, the e stands for encrypt. The k abc uses the word abc as its password.

```
$ echo 'hi hi hi ' | openssl enc -aes-128-cbc -e -k abc -pbkdf2 -a

U2FsdGVkX18DLd0bUBtfovtnDtRpLjku/GTZYe/wEIk=
```

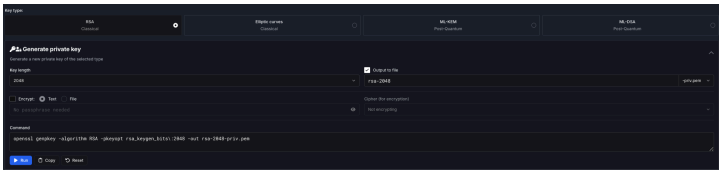
```
$ echo 'hi hi hi ' | openssl enc -aes-256-ctr -e -k abc -pbkdf2 -a

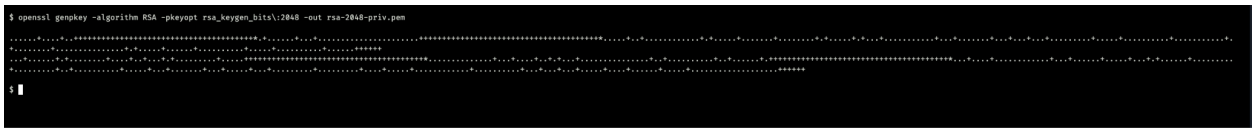
U2FsdGVkX1+wAEJZa4kCkt7hn2K4IUm2sHs=
```

```
$ echo 'hi hi hi ' | openssl enc -des -e -k abc -pbkdf2 -a

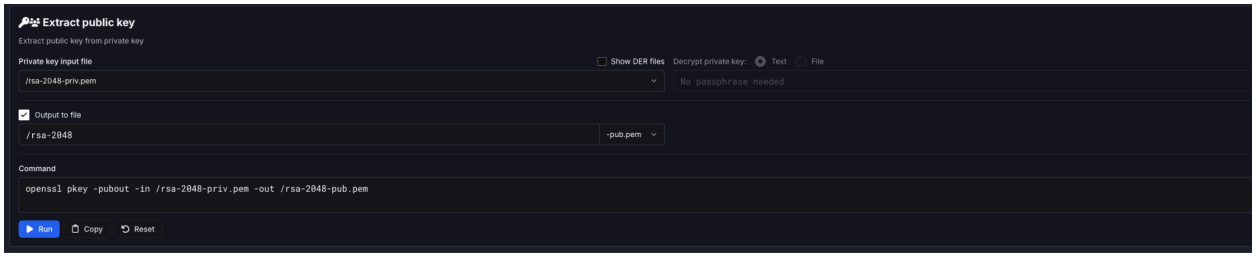
U2FsdGVkX18vFmq4K+TqtKMLpfEBpS0sreYwpzKHgKw=
```

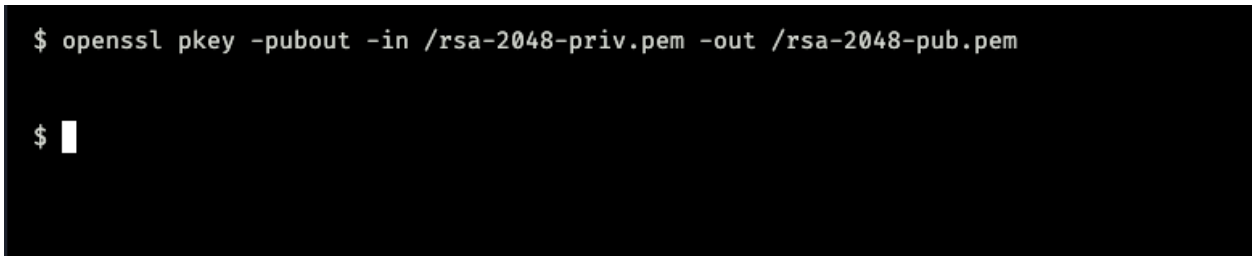
The aes-128-cbc encrypts using a 128 bit key and worlds in cbc mode which encrypts data in linked blocks. The AES 256 ctr uses a 256 bit key and works in the CTR mode which works as a stream cipher. DES uses a 56 bit key.

B) 



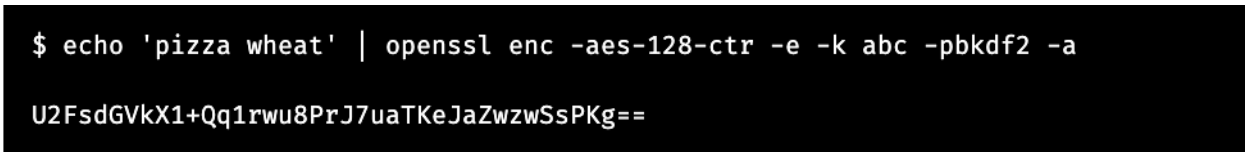
When creating the private key I used the tab at the bottom to help develop the command, genpkey is used to create a new private key. Following this is algorithm RSA which is an asymmetric algorithm, this is generating the type of key created. The next part is specifying how many bits for the key. The output are dots and pluses, the program is collecting randomness by generating large prime numbers.





When creating the public key, we use pkey which is used to process private keys and extract info to create the public keys.

4)

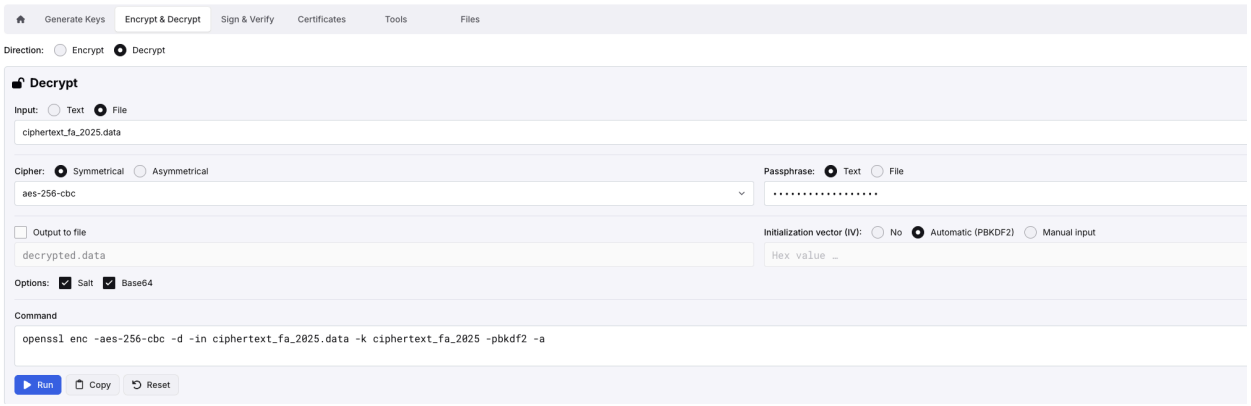


Creating my own encryption I used AES with a 128 bits (CTR) which is then encoded in Base64 with the password being abc.

c)







I struggled to decrypt the message, when creating the file I used the tab and selected the file I uploaded. However, I did not add the password to the passphrase, I still had it as the abc default which resulted in the output not being a language understood. Then I changed the passphrase which was then attached to the command which also output the same thing again. It wasn't until I completely deleted the k (key) that it was able to output the decrypted message, it was able to prompt the input of the password and the correct key was used - *"Thank you for your patience and cooperation for the receipt of this data. I hope you are having a good semester."*